

Informe de análisis y facturación – ImaxPrestamos

1. Resumen ejecutivo

- El backend PHP centraliza rutas en `api/index.php`, repite autenticación y mezcla controladores con consultas SQL directas, lo que complica el mantenimiento y la trazabilidad.
- El middleware de autenticación valida JWT manualmente (`api/middleware/auth.php`) con secretos embebidos en `config/config.php`, sin rotación ni almacenamiento seguro.
- El frontend es un conjunto de más de 40 HTML planos y respaldos `.backup` con el mismo layout duplicado (`frontend/index.html`, `frontend/usuarios.html`, etc.), estilos solo en `css/main.css` y lógica dispersa en archivos por vista dentro de `frontend/js/`.
- La base de datos depende de un único `schema_prestamos.sql` de 1.100 líneas con datos sembrados fijos; las migraciones sueltas en `database/migrations/` no están orquestadas.
- Existen únicamente 2 pruebas unitarias en `api/tests/`, por lo que no hay cobertura automatizada que garantice regresiones mínimas.

2. Supuestos para estimar

- **Equipo:** 1 dev backend senior + 1 dev frontend senior + 1 diseñador UI remoto. QA manual compartido.
- **Capacidad:** 40 h/semana por cada dev; diseñador trabaja por hitos dentro de la misma cadencia.
- **Tarifa mixta:** USD 45/hora (promedio entre backend, frontend y diseño).
- **Metodología:** Sprints de 1 semana; entregas parciales al cierre de cada hito crítico.

3. Roadmap por prioridad

Prioridad Alta (4.5 semanas · 180 h · USD 8,100)

Tarea	Alcance	Entregables clave	Duración	Presupuesto
H1. Endurecer autenticación y secretos	Extraer variables sensibles a <code>.env</code> , refactorizar <code>AuthMiddleware</code> para usar librerías JWT (base64url + firmas), agregar rotación de refresh tokens y cierre de sesión server-side.	Nueva capa <code>src/Core/Auth</code> , pruebas de integración para <code>/auth/*</code> , playbook de gestión de llaves.	1.5 semanas (60 h)	USD 2,700
H2. Reestructurar router y servicios backend	Dividir <code>api/index.php</code> en Router + Kernel, mover validaciones repetidas a <code>FormRequest</code> , aislar queries en <code>services/*</code> , documentar contratos en <code>docs/</code> .	Mini-framework interno (router + container), controladores del dominio préstamos/clientes reescritos con repositorios, cobertura de integración básica.	2 semanas (80 h)	USD 3,600

Tarea	Alcance	Entregables clave	Duración	Presupuesto
H3. Flujo completo de login/register	Revisar <code>frontend/js/auth.js</code> , garantizar sincronía con <code>AuthController</code> , añadir guardas de ruta y expiración silenciosa de sesión, mejorar feedback accesible en <code>login.html</code> y <code>register.html</code> .	Hooks <code>useSession</code> vanilla, componentes reutilizables de formularios, pruebas end-to-end de autenticación.	1 semana (40 h)	USD 1,800

Prioridad Media (3.5 semanas · 140 h · USD 6,300)

Tarea	Alcance	Entregables clave	Duración	Presupuesto
M1. Sistema de diseño + SASS	Convertir <code>css/main.css</code> en árbol SASS (<code>src/styles</code>), crear tokens (espaciado, color, tipografía), componentes para sidebar/header/cards; eliminar HTML duplicado consumiendo plantillas en <code>frontend/templates/</code> .	Design system documentado, build de estilos via Vite/Tailwind+SASS, kit Figma ligero.	1.5 semanas (60 h)	USD 2,700
M2. Modularizar frontend JS	Empaquetar <code>frontend/js/*.js</code> en módulos por dominio usando bundler (Vite/Rollup), crear store central y servicios compartidos (<code>apiClient</code> , <code>validators</code>), remover archivos <code>.backup</code> .	Árbol <code>src/modules</code> , build único minificado, pruebas de smoke con Playwright.	1 semana (40 h)	USD 1,800
M3. Gobernanza de base de datos	Normalizar <code>schema_prestamos.sql</code> en migraciones versionadas (Laravel/PHPStan migrator), separar seeds opcionales, añadir índices faltantes (pagos, auditoría), preparar scripts de rollback.	Pipeline <code>database/migrations</code> automatizado, checklist de despliegue, documentación de dependencias (DGII, JCE).	1 semana (40 h)	USD 1,800

Prioridad Baja (2 semanas · 80 h · USD 3,600)

Tarea	Alcance	Entregables clave	Duración	Presupuesto
L1. Automatización QA	Configurar PHPUnit + Pest para API y Vitest/Playwright para frontend, integrar cobertura mínima 60% en CI.	Workflow GitHub/GitLab CI, badges en README.	0.75 semanas (30 h)	USD 1,350

Tarea	Alcance	Entregables clave	Duración	Presupuesto
L2. Documentación operativa	Unificar README.md , INSTALACION.md , guías de WhatsApp y cooperativas; crear manual de despliegue y runbook de soporte.	Carpeta docs/ reorganizada, checklists PDF/Markdown para cliente final.	0.5 semanas (20 h)	USD 900
L3. Observabilidad y backups	Implementar logging estructurado (Monolog), métricas básicas (health-checks, job monitor cron/recordatorios), políticas de respaldo y retención.	Dashboards básicos (Grafana/Elastic), scripts de backup diario + restauración.	0.75 semanas (30 h)	USD 1,350

4. Cronograma sugerido

Semana	Hitos principales
1	Kickoff, hardening de auth (H1)
2	Router/Kernel backend (H2, parte 1)
3	Finalizar refactor backend y comenzar flujo auth completo (H2, parte 2 + H3)
4	Cerrar H3 y arrancar sistema de diseño (M1)
5	Finalizar M1 y modularización JS (M2)
6	Gobernanza DB (M3) y QA automático básico (L1, parte 1)
7	Documentación y observabilidad (L2, L3) + buffer para ajustes

Duración total estimada: **7 semanas calendario** (con traslape parcial de tareas de baja prioridad).

5. Riesgos y dependencias

- Acceso a credenciales reales (DGII, DataCréditos, WhatsApp) para validar integraciones; sin sandbox, los sprints de hardening podrían extenderse.
- Eliminación de archivos **.backup** y refactor de HTML requiere definir qué vistas permanecen; se sugiere sesión con stakeholders antes de M1.
- Cambios en esquema podrían requerir migraciones en caliente; planificar ventana de mantenimiento.

6. Próximos pasos recomendados

1. Aprobar este plan y el presupuesto (USD 18,000 total).
2. Definir ambientes (**dev**, **staging**, **prod**) y credenciales de despliegue.
3. Priorizar vistas fundamentales del frontend para diseñar el sistema de componentes.
4. Programar QA conjunto al finalizar cada sprint para mostrar avances al cliente.